

ARM PrimeCell Driver

common v1.1 (Common module)

API Specification

Primecell driver API description

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	B
Primecell driver identifier	common
Code Version	1.1

Contents

1	ABOUT THIS DOCUMENT	6
1.1	Change control	6
1.1.1	Change history	6
2	SUMMARY	7
3	DETAILS	8
4	RELEASED FILES	8
5	PUBLIC API (APTYPES.H)	9
5.1	Type Definitions	9
5.1.1	BOOL	9
5.1.2	BYTE8	9
5.1.3	HWD16	9
5.1.4	UBYTE8	9
5.1.5	UHWD16	9
5.1.6	UWORD32	9
5.1.7	WORD32	10
5.1.8	apTYPE_rCallback	10
5.2	Macros	10
5.2.1	CONST	10
5.2.2	FALSE	10
5.2.3	IGNORE	10
5.2.4	INLINE	10
5.2.5	MAX	10
5.2.6	MIN	11
5.2.7	PACKED	11
5.2.8	PRIVATE	11
5.2.9	PROTECTED	11
5.2.10	PUBLIC	11
5.2.11	TRUE	11
5.2.12	aNULL	12
5.2.13	aRNULL	12
5.2.14	apASSERT	12
5.3	Debug Macros	13
5.3.1	apDEBUG_CRIT	13
5.3.2	apDEBUG_INFO	14
5.3.3	apDEBUG_WARN	14
6	PUBLIC API (APBITOPS.H)	15
6.1	Macros	15
6.1.1	apBITS_ALL	15
6.1.2	apBIT_BUILD	15
6.1.3	apBIT_BUILD_FIELD	15
6.1.4	apBIT_CLEAR	15
6.1.5	apBIT_CLEAR_FIELD	16
6.1.6	apBIT_GET	16

6.1.7	apBIT_GET_FIELD	16
6.1.8	apBIT_GET_FIELD_UNSHIFTED	16
6.1.9	apBIT_GET_UNSHIFTED	17
6.1.10	apBIT_MASK	17
6.1.11	apBIT_MASK_FIELD	17
6.1.12	apBIT_SET	17
6.1.13	apBIT_SET_FIELD	18
6.1.14	apBIT_SET_FIELD_NOREAD	18
6.1.15	apBIT_SET_NOREAD	18
6.1.16	apBYTE_SET_FIELD	18
6.1.17	apBYTE_STREAM_ENDIAN_ADJUST	18
6.1.18	apHWD_STREAM_ENDIAN_ADJUST	19

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A	15/11/00	Initial draft, revised document to match desired template format

2 SUMMARY

These headers define types and macros used within the Primecell drivers

3 DETAILS

- ◆ Peak performance obtained in tests: N/A
- ◆ ROM size (RO code + RO data) (release build): 0 Bytes
- ◆ RAM size (ZI data) per instance (release build): 0 Bytes
- ◆ Does this driver support both endian-nesses?¹ YES
- ◆ Does this driver support Thumb?² YES

4 RELEASED FILES

PL000-ISPC-1004-B

This API document

apcommon/aptypes.h

Public header file defining basic types to match the coding standard

apcommon/apbitops.h

Public header file defining bit mask operations

¹ To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

² The driver may include ARM code, but this must interwork with a Thumb build.

5 PUBLIC API (APTYPES.H)

5.1 Type Definitions

5.1.1 BOOL

Primitive type – Boolean variable

Declaration

```
typedef unsigned long    BOOL
```

5.1.2 BYTE8

Primitive type – signed byte variable

Declaration

```
typedef signed char      BYTE8
```

5.1.3 HWD16

Primitive type – signed halfword variable

Declaration

```
typedef short            HWD16
```

5.1.4 UBYTE8

Primitive type – unsigned byte variable

Declaration

```
typedef unsigned char    UBYTE8
```

5.1.5 UHWD16

Primitive type – unsigned halfword variable

Declaration

```
typedef unsigned short    UHWD16
```

5.1.6 UWORD32

Primitive type – unsigned (32 bit) word variable

Declaration

```
typedef unsigned long     UWORD32
```

5.1.7 WORD32

Primitive type – (32 bit) word variable

Declaration

```
typedef long WORD32
```

5.1.8 apTYPE_rCallback

General function types

Declaration

```
typedef void (*apTYPE_rCallback) (UWORD32)
```

5.2 Macros

5.2.1 CONST

defines a constant item

Declaration

```
#define CONST const
```

5.2.2 FALSE

Declaration

```
#define FALSE (0)
```

5.2.3 IGNORE

indicates that the parameter is not used (for expansion)

Declaration

```
#define IGNORE(_v) ((void)(_v))
```

5.2.4 INLINE

specifies a routine to be in line

Declaration

```
#define INLINE __inline
```

5.2.5 MAX

The larger of __x and __y

Declaration

```
#define MAX(__x, __y) ((__x)>(__y)?(__x):(__y))
```

5.2.6 MIN

The smaller of __x and __y

Declaration

```
#define MIN(__x, __y) ((__x)<(__y)?(__x):(__y))
```

5.2.7 PACKED

defines a packed structure

Declaration

```
#define PACKED __packed
```

5.2.8 PRIVATE

an item with scope limited to the module

Declaration

```
#define PRIVATE static
```

5.2.9 PROTECTED

item has public scope but is not included in the API

Declaration

```
#define PROTECTED
```

5.2.10 PUBLIC

specifies an item of public API scope

Declaration

```
#define PUBLIC
```

5.2.11 TRUE

Declaration

```
#define TRUE (1)
```

5.2.12 aNULL

a null data pointer

Declaration

```
#define aNULL (void *) 0
```

5.2.13 aRNULL

a null pointer to a routine

Declaration

```
#define aRNULL aNULL
```

5.2.14 apASSERT

Declaration

```
#define apASSERT(__x)          assert(__x)
```

Remarks

```
#define apASSERT{xe "apASSERT"}(__x)
```

5.3 Debug Macros

The debug macros are controlled as follows:

- ◆ `apDEBUG` - if this is not defined, all debugs are disabled
- ◆ `apDEBUG_LEVEL` - this may be:
 - undefined / 0 : all debug macros used
 - 1 : only warning/critical debug macros enabled
 - 2 : only critical debug macros enabled
- ◆ `apDEBUG_BUFFER` - if this is defined, debugs are written to a buffer. Otherwise, debug data is written to `stderr`. If a debug buffer is used, the following variables **MUST** be declared in a source file.
 - `PUBLIC int apDEBUG_Counter=0;`
 - `PUBLIC char apDEBUG_Working[apDEBUG_PRINTF_MAX];`
 - `PUBLIC char apDEBUG_Buffer[apDEBUG_BUFFER_SIZE]="<#>";`
- ◆ `apDEBUG_BUFFER_SIZE` - if a debug buffer is used, this defines the size (default 1024).
- ◆ `apDEBUG_PRINTF_MAX` - if a debug buffer is used, this defines the largest size of a `printf()`. There is no checking that this is not exceeded. (default 128)

The macros are as follows:

- ◆ `apDEBUG_ENABLED` - this constant is used internally. If defined, the code is running in debug mode.
- ◆ `apDEBUG_INFO(__s,__d)` - Used to output debug information. The two arguments are passed to `printf`. Typical usage: `apDEBUG_INFO("Current status: %x\n",StatusWord);`
- ◆ `apDEBUG_WARN(__s,__d)` - Used to output warning information. The two arguments are passed to `printf`.
- ◆ `apDEBUG_CRIT(__s,__d)` - Used to output critical information. The two arguments are passed to `printf`.

When a buffer is specified, the data is written into the buffer. Data is written into consecutive locations in the buffer, returning to the start if the buffer is full. No debug message is ever split, although a long message may be truncated by the end of the buffer. The end of the latest message is marked by the token '`<#>`'.

5.3.1 `apDEBUG_CRIT`

Used to output critical debug information. The two arguments are passed to `printf`.

Typical usage: `apDEBUG_CRIT("Test Failed: %x\n",StatusWord);`

Declaration

```
#define apDEBUG_CRIT(__s,__d)
```

5.3.2 apDEBUG_INFO

Used to output debug information. The two arguments are passed to printf.

Typical usage: `apDEBUG_INFO("Current status: %x\n", StatusWord);`

Declaration

```
#define apDEBUG_INFO(__s, __d)
```

5.3.3 apDEBUG_WARN

Used to output warning debug information. The two arguments are passed to printf.

Typical usage: `apDEBUG_WARN("Incorrect parameter: %x\n", ParamValue);`

Declaration

```
#define apDEBUG_WARN(__s, __d)
```

6 PUBLIC API (APBITOPS.H)

6.1 Macros

6.1.1 apBITS_ALL

Generic constant for accessing all bits in a 32 bit word

Declaration

```
#define apBITS_ALL 0xFFFFFFFF
```

6.1.2 apBIT_BUILD

Shift the specified value into the desired bits

Declaration

```
#define apBIT_BUILD(__bws, __val) (((UWORD32)((UWORD32)(__val) << (UWORD32)(bs ## __bws)) & ((UWORD32) apBIT_MASK(__bws))))
```

Remarks

__bws a width/shift pair.

__val the data value to be shifted into the specified bits.

Returns The value shifted to the specified offset

6.1.3 apBIT_BUILD_FIELD

Equivalent to apBIT_BUILD{xe "apBIT_BUILD"}, but using separate width and shift parameters.

Declaration

```
#define apBIT_BUILD_FIELD(__bw, __bs, __val) (((UWORD32)((__val) << (__bs)) & ((UWORD32) apBIT_MASK_FIELD(__bw, __bs))))
```

6.1.4 apBIT_CLEAR

Clear the specified bits

Declaration

```
#define apBIT_CLEAR(__datum, __bws) ((__datum) = ((__datum) & ~((UWORD32) apBIT_MASK(__bws))))
```

Remarks

__datum the word of data to be modified

__bws a width/shift pair.

6.1.5 apBIT_CLEAR_FIELD

Equivalent to apBIT_CLEAR{xe "apBIT_CLEAR"}, but using separate width and shift parameters.

Declaration

```
#define apBIT_CLEAR_FIELD(__datum, __bw, __bs) (((__datum) = ((__datum) & ~((UWORD32) apBIT_MASK_FIELD(__bw, __bs))))
```

6.1.6 apBIT_GET

Access the specified bits from a word of data as an integer value

Declaration

```
#define apBIT_GET(__datum, __bws) ((UWORD32)(((__datum) & ((UWORD32) apBIT_MASK(__bws))) >> (bs ## __bws)))
```

Remarks

`__datum` the word of data to be accessed
`__bws` a width/shift pair.

Returns The relevant bits masked from the data word shifted to bit zero

6.1.7 apBIT_GET_FIELD

Equivalent to apBIT_GET{xe "apBIT_GET"}, but using separate width and shift parameters.

Declaration

```
#define apBIT_GET_FIELD(__datum, __bw, __bs) ((UWORD32)(((__datum) & ((UWORD32) apBIT_MASK_FIELD(__bw, __bs))) >> (__bs)))
```

6.1.8 apBIT_GET_FIELD_UNSHIFTED

Equivalent to apBIT_GET_UNSHIFTED{xe "apBIT_GET_UNSHIFTED"}, but using separate width and shift parameters.

Declaration

```
#define apBIT_GET_FIELD_UNSHIFTED(__datum, __bw, __bs) ((UWORD32)((__datum) & ((UWORD32) apBIT_MASK_FIELD(__bw, __bs))))
```

6.1.9 apBIT_GET_UNSHIFTED

Access the specified bits from a word of data at their original offset

Declaration

```
#define apBIT_GET_UNSHIFTED(__datum, __bws) (((__datum) & ((UWORD32) apBIT_MASK(__bws))))
```

Remarks

`__datum` the word of data to be accessed
`__bws` a width/shift pair.

Returns The relevant bits masked from the data word, still at their original offset

6.1.10 apBIT_MASK

Build a mask for the specified bits

Declaration

```
#define apBIT_MASK(__bws) ((UWORD32)((bw ## __bws)==32)?0xFFFFFFFF:((1U << (bw ## __bws)) - 1) << (bs ## __bws))
```

Remarks

`__bws` a width/shift pair.

Returns a mask with the bits to be addressed set and all others cleared

6.1.11 apBIT_MASK_FIELD

Equivalent to `apBIT_MASK{xe "apBIT_MASK"}`, but using separate width and shift parameters.

Declaration

```
#define apBIT_MASK_FIELD(__bw, __bs) ((UWORD32)((__bw)==32)?0xFFFFFFFF:(UWORD32)((UWORD32)1U << (__bw)) - 1) << (__bs))
```

6.1.12 apBIT_SET

Place the specified value into the specified bits of a word of data

Declaration

```
#define apBIT_SET(__datum, __bws, __val) ((__datum) = ((UWORD32) (__datum) & (UWORD32)~(apBIT_MASK(__bws))) | \ ((UWORD32) ((UWORD32)(__val) << (UWORD32)(bs ## __bws)) & (apBIT_MASK(__bws))))
```

Remarks

`__datum` the word of data to be accessed
`__bws` a width/shift pair.
`__val` the data value to be shifted into the specified bits.

6.1.13 apBIT_SET_FIELD

Equivalent to apBIT_SET{xe "apBIT_SET"}, but using separate width and shift parameters.

Declaration

```
#define apBIT_SET_FIELD(__datum, __bw, __bs, __val) ((__datum) = ((__datum) &
~((UWORD32) apBIT_MASK_FIELD(__bw, __bs))) | \
((__bs)) & ((UWORD32) apBIT_MASK_FIELD(__bw, __bs)))) ((UWORD32) ((__val) <<
```

6.1.14 apBIT_SET_FIELD_NOREAD

Equivalent to apBIT_SET_NOREAD{xe "apBIT_SET_NOREAD"}, but using separate width and shift parameters.

Declaration

```
#define apBIT_SET_FIELD_NOREAD(__datum, __bw, __bs, __val) (UWORD32) ((__datum) =
(((__val) << (__bs)) & ((UWORD32) apBIT_MASK_FIELD(__bw, __bs))))
```

6.1.15 apBIT_SET_NOREAD

Place the specified value into the specified bits of a word of data without reading first - for sensitive interrupt type registers

Declaration

```
#define apBIT_SET_NOREAD(__datum, __bws, __val) (UWORD32) ((__datum) = (((__val) <<
(bs ## __bws)) & ((UWORD32) apBIT_MASK(__bws))))
```

Remarks

<code>__datum</code>	the word of data to be accessed
<code>__bws</code>	a width/shift pair.
<code>__val</code>	the data value to be shifted into the specified bits.

6.1.16 apBYTE_SET_FIELD

Equivalent to apBIT_SET_FIELD{xe "apBIT_SET_FIELD"}, but with BYTE width parameters.

Declaration

```
#define apBYTE_SET_FIELD(__datum, __bw, __bs, __val) ((__datum) = (BYTE8)
(((__datum) & ~(apBIT_MASK_FIELD(__bw, __bs))) | (((__val) << (__bs)) &
(apBIT_MASK_FIELD(__bw, __bs))))
```

6.1.17 apBYTE_STREAM_ENDIAN_ADJUST

Declaration

```
#define apBYTE_STREAM_ENDIAN_ADJUST(_w)
```

Remarks

Required to reverse the FIFO order for the Primecells to match a big endian system architecture

Requires the `__BIG_ENDIAN` constant to be defined for a big-endian system

6.1.18 apHWD_STREAM_ENDIAN_ADJUST

Declaration

```
#define apHWD_STREAM_ENDIAN_ADJUST(_w)
```

Remarks

Required to reverse the FIFO order for the Primecells to match a big endian system architecture

Requires the __BIG_ENDIAN constant to be defined for a big-endian system